

Workshop 10: SQL Database Lab

PostgreSQL, CRUD และ Database Integration

เป้าหมาย

Workshop นี้ให้นักศึกษาฝึกใช้งานฐานข้อมูล PostgreSQL และเชื่อมต่อฐานข้อมูลเข้ากับ Node.js และ Express

โดยเน้น

- Database, Table, Row และ Column
- Primary Key
- SQL CRUD
- SELECT และ WHERE
- PostgreSQL + Node.js + Express
- Connection Pool
- Parameterized Query
- REST API และ Database

Workshop ประกอบด้วย 3 งาน

1. PostgreSQL CRUD Basics
2. Connect Express to PostgreSQL
3. Complete CRUD API

งานที่ 1 — PostgreSQL CRUD Basics

สร้างฐานข้อมูลและ Table สำหรับเก็บข้อมูลผู้ใช้

Step 1 — สร้าง Database

ใช้คำสั่ง

```
CREATE DATABASE mydatabase;
```

Step 2 — สร้าง Table

สร้าง Table ชื่อ

users

โดยประกอบด้วย

- id — Primary Key
- name — ชื่อผู้ใช้
- age — อายุ

ใช้คำสั่ง

```
CREATE TABLE users (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    age INT  
);
```

Step 3 — INSERT

เพิ่มข้อมูลลงใน Table

```
INSERT INTO users (name, age)  
VALUES ('Alice', 25);  
  
INSERT INTO users (name, age)  
VALUES  
    ('Bob', 30),  
    ('Charlie', 35);
```

Step 4 — SELECT

ดูข้อมูลทั้งหมด

```
SELECT * FROM users;
```

เลือกเฉพาะบาง Column

```
SELECT name, age FROM users;
```

Step 5 — WHERE

ค้นหาผู้ใช้ที่อายุมากกว่า 25 ปี

```
SELECT * FROM users
WHERE age > 25;
```

ค้นหาผู้ใช้ชื่อ Alice

```
SELECT * FROM users
WHERE name = 'Alice';
```

Step 6 — UPDATE

แก้ไขอายุของ User ที่มี id = 1

```
UPDATE users
SET age = 26
WHERE id = 1;
```

Step 7 — DELETE

ลบ User ที่มี id = 1

```
DELETE FROM users
WHERE id = 1;
```

ข้อควรระวัง:

ไม่ควรใช้

```
DELETE FROM users;
```

หากไม่ต้องการลบข้อมูลทั้งหมดใน Table

งานที่ 2 — Connect Express to PostgreSQL

นำ Project Express จาก Workshop ก่อนหน้า มาเชื่อมต่อกับ PostgreSQL
โครงสร้าง Project

```
myapp/  
|-- server.js  
|-- package.json  
|  
`-- public/  
      |-- index.html
```

Step 1 — ติดตั้ง PostgreSQL Driver

เปิด Terminal แล้วพิมพ์

```
npm install pg
```

Step 2 — สร้าง Connection Pool

ในไฟล์ server.js

```
const express = require("express");  
const { Pool } = require("pg");  
  
const app = express();  
  
const pool = new Pool({  
  user: "YOUR_USER",  
  host: "localhost",  
  database: "mydatabase",  
  password: "YOUR_PASSWORD",  
  port: 5432  
});
```

ให้นักศึกษาเปลี่ยน

YOUR_USER

YOUR_PASSWORD

ให้ตรงกับ PostgreSQL ของเครื่องตนเอง

Step 3 — Middleware

เพิ่ม

```
app.use(express.json());

app.use(
  express.urlencoded({
    extended: true
  })
);
```

Middleware นี้ช่วยแปลงข้อมูลจาก Request ให้อยู่ในรูปแบบ JavaScript Object

Step 4 — GET ข้อมูลจาก Database

สร้าง Route

```
app.get("/users", async (req, res) => {

  const result =
    await pool.query(
      "SELECT * FROM users"
    );

  res.json(result.rows);

});
```

จากนั้น Run Server

```
node server.js
```

ทดลองเปิด

<http://localhost:3000/users>

ควรได้รับข้อมูลจาก PostgreSQL ในรูปแบบ JSON

งานที่ 3 — Complete CRUD API

เพิ่ม REST API สำหรับ

Create, Read, Update และ Delete

โดยให้ Express ทำหน้าที่เป็นตัวกลาง ระหว่าง Client และ PostgreSQL

1. READ — GET

```
app.get("/users", async (req, res) => {  
  
  const result =  
    await pool.query(  
      "SELECT * FROM users"  
    );  
  
  res.json(result.rows);  
  
});
```

2. CREATE — POST

รับข้อมูล

- name
- age

จาก req.body

```
app.post("/users", async (req, res) => {  
  
  const { name, age } = req.body;  
  
  const result = await pool.query(  
    "INSERT INTO users (name, age)  
    VALUES ($1, $2)  
    RETURNING *",  
    [name, age]  
  );  
  
  res.status(201).json(result.rows[0]);  
  
});
```

```
});
```

Parameterized Query

ในคำสั่ง

```
VALUES ($1, $2)
```

ค่าจริงจะถูกส่งแยกออกมาเป็น

```
[name, age]
```

วิธีนี้ปลอดภัยกว่าการนำข้อมูลของผู้ใช้ มาต่อกับ SQL Command โดยตรง และช่วยป้องกัน SQL Injection

3. UPDATE — PUT

ใช้ :id สำหรับระบุ User ที่ต้องการแก้ไข

```
app.put("/users/:id", async (req, res) => {

  const id = req.params.id;
  const { name, age } = req.body;

  const result = await pool.query(
    "UPDATE users
     SET name = $1, age = $2
     WHERE id = $3
     RETURNING *",
    [name, age, id]
  );

  res.json(result.rows[0]);

});
```

4. DELETE

```
app.delete("/users/:id", async (req, res) => {

  const id = req.params.id;
```

```
const result = await pool.query(  
  "DELETE FROM users  
  WHERE id = $1  
  RETURNING *",  
  [id]  
);  
  
res.json(result.rows[0]);  
  
});
```

ความสัมพันธ์ระหว่าง HTTP และ SQL

งาน	HTTP	SQL
Create	POST	INSERT
Read	GET	SELECT
Update	PUT	UPDATE
Delete	DELETE	DELETE

Challenge — เรียก API จาก Frontend

ในไฟล์

public/index.html

สร้างปุ่ม

```
<button onclick="loadUsers()">
  Load Users
</button>

<div id="result"></div>
```

เพิ่ม JavaScript

```
async function loadUsers() {

  const response =
    await fetch("/users");

  const users =
    await response.json();

  console.log(users);

}
```

Flow การทำงานคือ

Frontend → **fetch()** → **Express** → **SQL** → **PostgreSQL**

จากนั้นข้อมูลจะถูกส่งกลับเป็น JSON มายัง Frontend

Checklist ก่อนส่งงาน

งานที่ 1 — PostgreSQL

- ☐ สร้าง Database mydatabase
- ☐ สร้าง Table users
- ☐ มี Primary Key
- ☐ ใช้ INSERT ได้
- ☐ ใช้ SELECT ได้
- ☐ ใช้ WHERE ได้
- ☐ ใช้ UPDATE ได้
- ☐ ใช้ DELETE ได้

งานที่ 2 — Database Connection

- ☐ ติดตั้ง Package pg
- ☐ สร้าง Connection Pool
- ☐ ใช้ `express.json()`
- ☐ Express เชื่อมต่อ PostgreSQL ได้
- ☐ GET /users แสดงข้อมูลได้

งานที่ 3 — CRUD API

- ☐ GET ใช้ SELECT
- ☐ POST ใช้ INSERT
- ☐ PUT ใช้ UPDATE
- ☐ DELETE ใช้ DELETE
- ☐ ใช้ Parameterized Query
- ☐ รับ id ผ่าน `req.params`
- ☐ ส่งข้อมูลกลับเป็น JSON

งานที่นักศึกษาต้องส่ง

ให้นักศึกษาส่ง

1. ไฟล์ SQL ที่ใช้สร้างและจัดการ Table
2. Folder Project Node.js / Express
3. ไฟล์ `server.js`
4. Screenshot ผลลัพธ์ของ GET, POST, PUT และ DELETE

Frontend → Express → SQL → PostgreSQL